Software Re-Engineering

Lecture 17

Reverse engineering III – IDA Navigation

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Readings

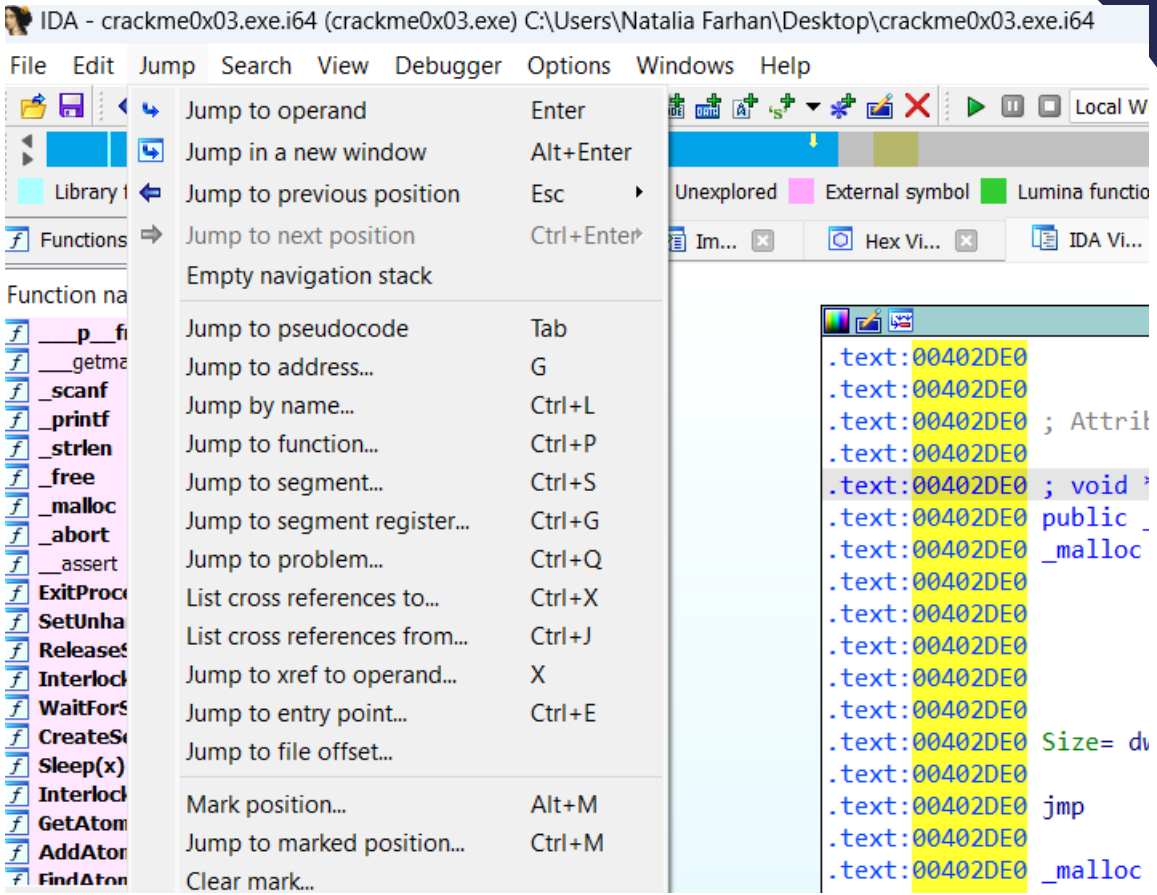
Chapter 6: DISASSEMBLY NAVIGATION, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle

Double-Click Navigation

- When a program is disassembled, every location in the program is assigned a virtual address. As a result, we can navigate anywhere within a program by providing the virtual address of the location we are interested in visiting.
- Unfortunately for us, maintaining a catalog of addresses in our head is not a trivial task
- IDA generates symbolic names during the analysis phase by examining a binary’s symbol table or by automatically generating a name based on how a location is referenced within the binary.

Jump to Address

Ultimately, the easiest way to get to a known disassembly location is to make use of the **Jump to Address** dialog



Navigation History

- Another feature IDA shares with traditional web browsers is the concept of forward and backward navigation based on the order in which you navigate the disassembly.
- Each time you navigate to a new location within a disassembly, your current location is appended to a history list.
- **Jump->Jump to Previous Position** repositions the disassembly to the most recent entry in the history list

- **Jump->Jump to Next Position** is the counterpart operation that moves the disassembly window forward in the history list in a manner similar to a web browser's forward button.

two of the more useful toolbar buttons provide the familiar browser-style forward and backward behavior



Because IDA Pro is such a low-level analysis tool, many of its features and displays expect the user to be somewhat familiar with the low-level details of compiled languages

Stack Frames

- Stack frames are blocks of memory allocated within a program’s runtime stack and dedicated to a specific invocation of a function.
- Programmers typically group executable statements into units called functions

When a function is not executing, it typically requires little to no memory. When a function is called, however, it may require memory for several reasons. First, the caller of a function may wish to pass information into the function in the form of parameters (arguments), and these parameters need to be stored somewhere the function can find them. Second, the function may need temporary storage space while performing its task. This temporary space is often allocated by a programmer through the declaration of local variables, which can be used within the function but cannot be accessed once the function has completed.

**RECALL

- The register ‘ESP’ is used to point to the next item on the stack and is referred to as the ‘stack pointer’.
- EBP aka the ‘frame pointer’ serves as an unchanging reference point for data on the stack.
- This allows the program to work out how far away something in the stack is from this point. So if a variable is two ‘building blocks’ away then it is [EBP+8] as each ‘block’ in the stack is 4 bytes.
- Each function in a program will generate its own stack frame to reference its own variables and parameters

The Building Blocks Of Stack Memory



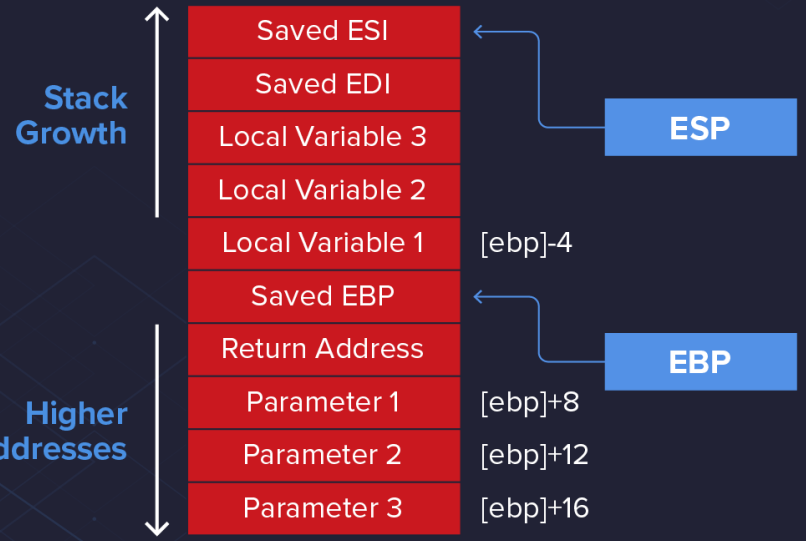
Stack frames are the foundation of stack memory.



ESB, aka "frame pointers," are an unchanging reference point for data on the stack.



ESP, aka "stack pointers," point to the next item on the stack.



VARONIS

- Lower memory addresses are on top and higher memory addresses are on the bottom.
- Each function will generate its own stack frame, so the stack frame in the example above could be on top of another frame on the stack which is being used for another function.
- EBP as mentioned earlier is stored as an unchanging reference point on the stack, this is done by moving the value of ESP (the stack pointer) into EBP. We do this as ESP will change as it always points to the top of the stack, storing it in EBP gives us an unchanging reference point in the stack and now the function can reference its variables and parameters on the stack from this location.
- In this example parameters that have been passed to the function are stored in '[EBP]+8', '[EBP]+12' and '[EBP] +16'. So when we see '[EBP]+8' that is the distance on the stack from EBP.
- Variables will be stored after the function has begun executing so these will be stored higher up the stack but in lower address space, so in this example, this will be shown as '[EBP]-4'.

- Compilers utilize **stack frames (also called activation records)** to make the allocation and deallocation of function parameters and local variables transparent to the programmer.
- A **calling convention** dictates exactly where a caller should place any parameters that a function requires. Calling conventions may require parameters to be placed in specific registers, on the program stack, or in both registers and on the stack
- The default calling convention used by most C compilers for the x86 architecture is called the C calling convention.
- Calling conventions are often language-, compiler-, and CPU-specific

Consider calls to a function having the following prototype:

```
void demo_cdecl(int w, int x, int y, int z);
```

- By default, this function will use the **cdecl calling convention**, expecting the four parameters to be pushed in right-to-left order and requiring the caller to clean the parameters off the stack.
- A compiler might generate code for a call to this function as follows:

```
; demo_cdecl(1, 2, 3, 4);   //programmer calls demo_cdecl
❶ push   4           ; push parameter z
  push   3           ; push parameter y
  push   2           ; push parameter x
  push   1           ; push parameter w
  call   demo_cdecl  ; call the function
❷ add    esp, 16     ; adjust esp to its former value
```

The four push operations beginning at X result in a net change to the program stack pointer (ESP) of 16 bytes (4 * sizeof(int) on a 32-bit architecture), which is undone at Y following the return from demo_cdecl.

We compute the minimum amount of stack space required for local variables as 76 bytes (three 4-byte integers and a 64-byte buffer).

```
sub   esp, 76     ; allocate sufficient space for all local variables
```

```
void bar(int j, int k);   // a function to call
void demo_stackframe(int a, int b, int c) {
   int x;
   char buffer[64];
   int y;
   int z;
   // body of function not terribly relevant other than
   bar(z, y);
}
```

The **Offset column** indicates the base+displacement address required to reference any of the local variables or parameters in the stack frame.

	Variable	Offset	
esp →	z	[esp]	local variables
	y	[esp+4]	
	buffer	[esp+8]	
	x	[esp+72]	
	saved eip	[esp+76]	
	a	[esp+80]	parameters
	b	[esp+84]	
	c	[esp+88]	

8 - 4= 4 bytes

72 - 8= 64

76 - 72= 4

An ESP-based stack frame

```
void bar(int j, int k);   // a function to call
void demo_stackframe(int a, int b, int c) {
   int x;
   char buffer[64];
   int y;
   int z;
   // body of function not terribly relevant other than
   bar(z, y);
}
```

In x86 programs, the EBP (extended base pointer) register is typically dedicated for use as a stack frame pointer.

```
❸ push    ebp         ; save the caller's ebp value
❹ mov     ebp, esp    ; make ebp point to the saved register value
❺ sub     esp, 76     ; allocate space for local variables
```

An EBP-based stack frame

	Variable	Offset	
esp →	z	[ebp-76]	local variables
	y	[ebp-72]	
	buffer	[ebp-68]	
	x	[ebp-4]	
ebp →	saved ebp	[ebp]	saved register(s)
	saved eip	[ebp+4]	
	a	[ebp+8]	parameters
	b	[ebp+12]	
	c	[ebp+16]	

An ESP-based stack frame

	Variable	Offset	
esp →	z	[esp]	local variables
	y	[esp+4]	
	buffer	[esp+8]	
	x	[esp+72]	
	saved eip	[esp+76]	
	a	[esp+80]	parameters
	b	[esp+84]	
	c	[esp+88]	

The call to function bar can now be implemented as follows

```
❻ push   dword [ebp-72]        ; push y
  push   dword [ebp-76]        ; push z
  call   bar
  add    esp, 8                ; cdecl requires caller to clear parameters
```

In x86 programs utilizing EBP as a frame pointer, the following code represents a typical epilogue:

```
mov     esp, ebp        ; clears local variables by reseting esp
pop     ebp             ; restore the caller's value of ebp
ret                     ; pop return address to return to the caller
```

This operation is so common that the x86 architecture offers the **leave instruction** as an abbreviated means of accomplishing the same task

```
leave                   ; copies ebp to esp AND then pops into ebp
ret                     ; pop return address to return to the caller
```

IDA Stack Views

- IDA offers two views into any function’s stack frame: a summary view and a detail view.
- In order to understand these two views, we will refer to the following version of demo_stackframe, which we have compiled using gcc.

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

In this example, local variables x and y are initialized from parameters c and b, respectively. Local variable z is initialized with the constant value 10, and the first character in the 64-byte local array, named buffer, is initialized to the letter 'A'.

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

The corresponding IDA disassembly of this function appears here.

```
.text:00401090 ; ========= S U B R O U T I N E ============================
.text:00401090
.text:00401090 ; Attributes: ❶bp-based frame
.text:00401090
.text:00401090 demo_stackframe proc near      ; CODE XREF: sub_4010C1+41↓p
.text:00401090
❹ .text:00401090 var_60          = dword ptr -60h
.text:00401090 var_5C          = dword ptr -5Ch
.text:00401090 var_58          = byte ptr -58h
.text:00401090 var_C           = dword ptr -0Ch
.text:00401090 arg_4           = dword ptr  0Ch
.text:00401090 arg_8           = dword ptr  10h
.text:00401090
.text:00401090                 push    ebp
.text:00401091                 mov     ebp, esp
.text:00401093                 sub     esp, ❷78h
.text:00401096                 mov     eax, [ebp+❺arg_8]
.text:00401099                 ❻mov    [ebp+var_C], eax
.text:0040109C                 ❼mov    eax, [ebp+arg_4]
.text:0040109F                 ❼mov    [ebp+var_5C], eax
.text:004010A2                 ❽mov    [ebp+var_60], 0Ah
.text:004010A9                 ❾mov    [ebp+var_58], 41h
.text:004010AD                   mov    eax, [ebp+var_5C]
.text:004010B0                 ❸mov    [esp+4], eax
.text:004010B4                   mov    eax, [ebp+var_60]
.text:004010B7                 ❸mov    [esp], eax
.text:004010BA                   call   bar
.text:004010BF                   leave
.text:004010C0                   retn
.text:004010C0 demo_stackframe endp
```

```
.text:00401090 ; ========= S U B R O U T I N E ============================
.text:00401090
.text:00401090 ; Attributes: ❶bp-based frame
.text:00401090
.text:00401090 demo_stackframe proc near      ; CODE XREF: sub_4010C1+41↓p
.text:00401090
❹ .text:00401090 var_60          = dword ptr -60h
.text:00401090 var_5C          = dword ptr -5Ch
.text:00401090 var_58          = byte ptr -58h
.text:00401090 var_C           = dword ptr -0Ch
.text:00401090 arg_4           = dword ptr  0Ch
.text:00401090 arg_8           = dword ptr  10h
.text:00401090
.text:00401090                 push    ebp
.text:00401091                 mov     ebp, esp
.text:00401093                 sub     esp, ❷78h
.text:00401096                 mov     eax, [ebp+❺arg_8]
```

We begin at (1) by noting that IDA believes this function uses the EBP register as a frame pointer based on analysis of the function prologue

```
.text:00401090                push    ebp
.text:00401091                mov     ebp, esp
.text:00401093                sub     esp, ❷78h
.text:00401096                mov     eax, [ebp+❺arg_8]
```

At (2) we learn that gcc has allocated 120 bytes (78h equates to 120 = 7x16^1+8x16^0) of local variable space in the stack frame.

This includes 8 bytes for passing the two parameters to bar at Z some bytes that compiler occasionally pad the local variable space with extra bytes in order to ensure a particular alignment within the stack frame.

```
.text:00401090 ; ========= S U B R O U T I N E ============================
.text:00401090
.text:00401090 ; Attributes: ❶bp-based frame
.text:00401090
.text:00401090 demo_stackframe proc near      ; CODE XREF: sub_4010C1+41↓p
.text:00401090
❹ .text:00401090 var_60          = dword ptr -60h
.text:00401090 var_5C          = dword ptr -5Ch
.text:00401090 var_58          = byte ptr -58h
.text:00401090 var_C           = dword ptr -0Ch
.text:00401090 arg_4           = dword ptr  0Ch
.text:00401090 arg_8           = dword ptr  10h
.text:00401090
```

Beginning at (4), IDA provides a summary stack view that lists every variable that is directly referenced within the stack frame, along with the variable's size and offset distance from the frame pointer

- IDA assigns names to variables based on their location relative to the saved return address.
- Local variables lie above the saved return address, while function parameters lie below the saved return address.

- Local variable names are derived using the **var**_ prefix joined with a **hexadecimal suffix** that indicates the distance, in bytes, that the variable lies above the saved frame pointer.
- Local variable **var_C**, in this case, is a 4-byte (dword) variable that lies **12 bytes** above the saved frame pointer **([ebp-0Ch])**

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

- Function parameter names are generated using the **arg**_ prefix combined with a **hexadecimal suffix** that represents the relative distance from the topmost parameter.
- Thus the topmost 4-byte parameter would be named **arg_0**, while successive parameters would be named **arg_4, arg_8, arg_C**, and so on

In this particular example arg_0 is not listed because the function makes no use of parameter a. Because IDA fails to locate any memory reference to [ebp+8] (the location of the first parameter), arg_0 is not listed in the summary stack view. A quick scan of the summary stack view reveals that there are many stack locations that IDA has failed to name because no direct references to those locations exist in the program code.

The three parameters: a, b, and c correspond to variables arg_0, arg_4, and arg_8 respectively (though arg_0 is missing in the disassembly because it is never referenced).

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

```
.text:00401090                 push    ebp
.text:00401091                 mov     ebp, esp
.text:00401093                 sub     esp, ❷78h
.text:00401096                 mov     eax, [ebp+❺arg_8]
.text:00401099                 ❻mov     [ebp+var_C], eax
.text:0040109C                 ❼mov     eax, [ebp+arg_4]
.text:0040109F                 ❼mov     [ebp+var_5C], eax
.text:004010A2                 ❽mov     [ebp+var_60], 0Ah
.text:004010A9                 ❾mov     [ebp+var_58], 41h
.text:004010AD                   mov     eax, [ebp+var_5C]
.text:004010B0                 ❸mov     [esp+4], eax
.text:004010B4                   mov     eax, [ebp+var_60]
.text:004010B7                 ❸mov     [esp], eax
.text:004010BA                   call    bar
.text:004010BF                   leave
.text:004010C0                   retn
.text:004010C0 demo_stackframe endp
```

Local variable x is initialized from parameter c. Thus **var_C** corresponds to **x** since it is initialized from arg_8 at].

var_ 5C=92
var_ 60=96
var_ 58=88

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

int x = c;

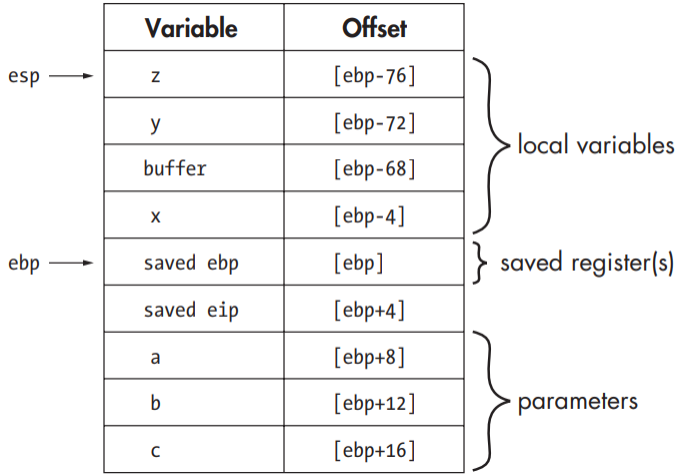
ebp+12



```
.text:00401090                  push    ebp
.text:00401091                  mov     ebp, esp
.text:00401093                  sub     esp, ❷78h
.text:00401096                  mov     eax, [ebp+❺arg_8]
.text:00401099                  ❻mov    [ebp+var_C], eax
.text:0040109C                  ❼mov    eax, [ebp+arg_4]
.text:0040109F                  ❼mov    [ebp+var_5C], eax
.text:004010A2                  ❽mov    [ebp+var_60], 0Ah
.text:004010A9                  ❾mov    [ebp+var_58], 41h
.text:004010AD                    mov   eax, [ebp+var_5C]
.text:004010B0                  ❸mov    [esp+4], eax
.text:004010B4                    mov   eax, [ebp+var_60]
.text:004010B7                  ❸mov    [esp], eax
.text:004010BA                    call  bar
.text:004010BF                    leave
.text:004010C0                    retn
.text:004010C0 demo_stackframe endp
```

The three parameters: a, b, and c correspond to variables arg_0, arg_4, and arg_8 respectively (though arg_0 is missing in the disassembly because it is never referenced).

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
```



	Variable	Offset	
esp →	z	[ebp-76]	local variables
	y	[ebp-72]	
	buffer	[ebp-68]	
	x	[ebp-4]	
ebp →	saved ebp	[ebp]	saved register(s)
	saved eip	[ebp+4]	
	a	[ebp+8]	parameters
	b	[ebp+12]	
	c	[ebp+16]	

```
.text:00401090                  push    ebp
.text:00401091                  mov     ebp, esp
.text:00401093                  sub     esp, ❷78h
.text:00401096                  mov     eax, [ebp+❺arg_8]
.text:00401099                  ❻mov    [ebp+var_C], eax
.text:0040109C  int y = b;      ❼mov    eax, [ebp+arg_4]
.text:0040109F                  ❼mov    [ebp+var_5C], eax
.text:004010A2  ebp+92          ❽mov    [ebp+var_60], 0Ah
.text:004010A9                  ❾mov    [ebp+var_58], 41h
.text:004010AD                  mov     eax, [ebp+var_5C]
.text:004010B0                  ❸mov    [esp+4], eax
.text:004010B4                  mov     eax, [ebp+var_60]
.text:004010B7                  ❸mov    [esp], eax
.text:004010BA                  call    bar
.text:004010BF                  leave
.text:004010C0                  retn
.text:004010C0 demo_stackframe endp
```

- Local variable z corresponds to var_60 since it is initialized with the value 10
- The 64-byte character array buffer begins at var_58 since buffer[0] is initialized with A (ASCII 0x41)

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

```
.text:00401090                  push    ebp
.text:00401091                  mov     ebp, esp
.text:00401093                  sub     esp, ❷78h
.text:00401096                  mov     eax, [ebp+❺arg_8]
.text:00401099 int z=10;        ❻mov     [ebp+var_C], eax
.text:0040109C buffer[0] = 'A'; ❼mov     eax, [ebp+arg_4]
.text:0040109F                  ❼mov     [ebp+var_5C], eax
.text:004010A2    ebp+96        ❽mov     [ebp+var_60], 0Ah
.text:004010A9    ebp+88        ❾mov     [ebp+var_58], 41h
.text:004010AD                    mov     eax, [ebp+var_5C]
.text:004010B0                  ❸mov     [esp+4], eax
.text:004010B4                    mov     eax, [ebp+var_60]
.text:004010B7                  ❸mov     [esp], eax
.text:004010BA                    call    bar
.text:004010BF                    leave
.text:004010C0                    retn
.text:004010C0 demo_stackframe endp
```

The two arguments for the call to bar are moved into the stack

```
void demo_stackframe(int a, int b, int c) {
 int x = c;
 char buffer[64];
 int y = b;
 int z = 10;
 buffer[0] = 'A';
 bar(z, y);
}
```

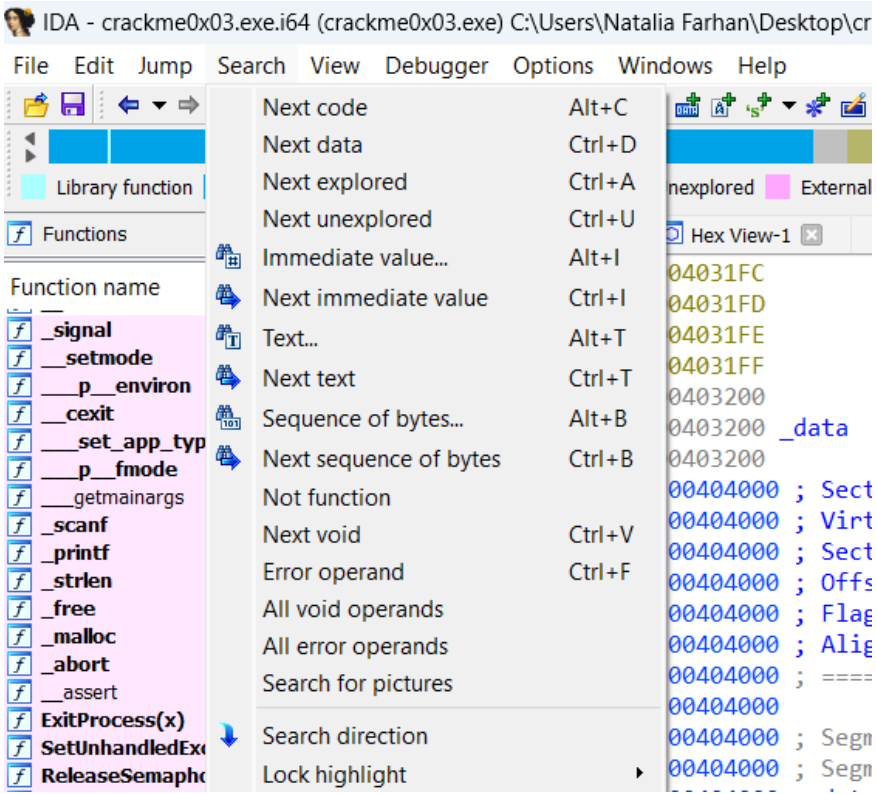
```
.text:00401090                 push    ebp
.text:00401091                 mov     ebp, esp
.text:00401093                 sub     esp, ❷78h
.text:00401096                 mov     eax, [ebp+❺arg_8]
.text:00401099                ❻mov     [ebp+var_C], eax
.text:0040109C                ❼mov     eax, [ebp+arg_4]
.text:0040109F                ❼mov     [ebp+var_5C], eax
.text:004010A2                ❽mov     [ebp+var_60], 0Ah
.text:004010A9                ❾mov     [ebp+var_58], 41h
.text:004010AD                 mov     eax, [ebp+var_5C]
.text:004010B0                ❸mov     [esp+4], eax
.text:004010B4                 mov     eax, [ebp+var_60]
.text:004010B7                ❸mov     [esp], eax
.text:004010BA                 call    bar
.text:004010BF                 leave
.text:004010C0                 retn
.text:004010C0 demo_stackframe endp
```

Searching the Database

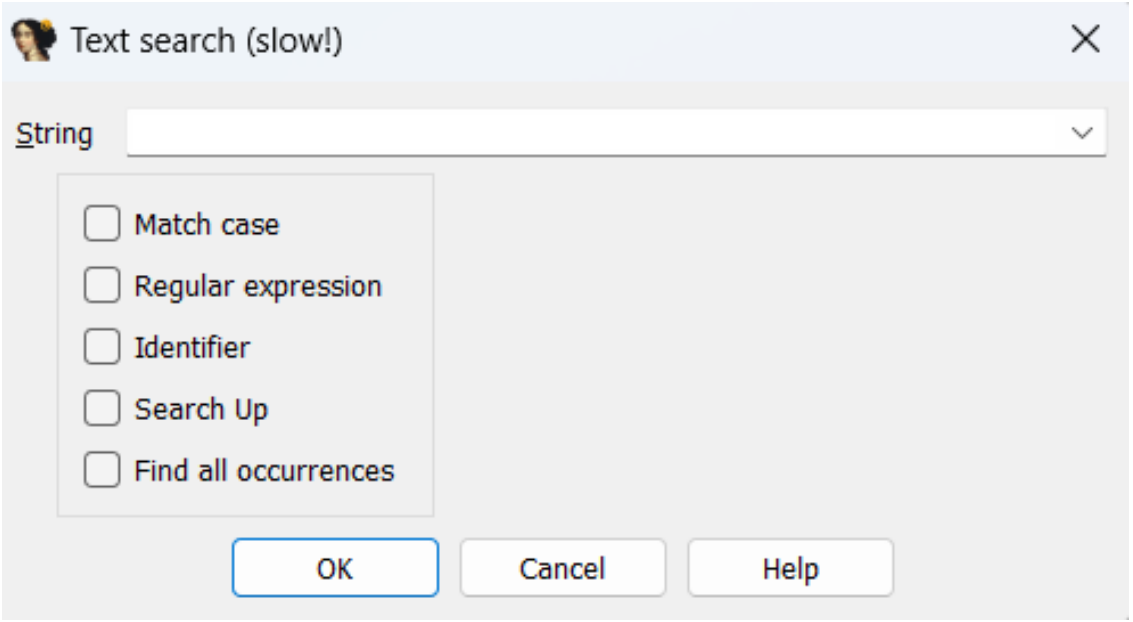
- **Search->Next Code** moves the cursor to the next location containing an instruction.
- **Jump->Jump to Function** brings up a list of all functions, allowing you to quickly choose one and navigate to it.
- While these canned search features may often be useful, two types of general-purpose searches are worth more detailed discussion: **text searches** and **binary searches.**

Text Searches

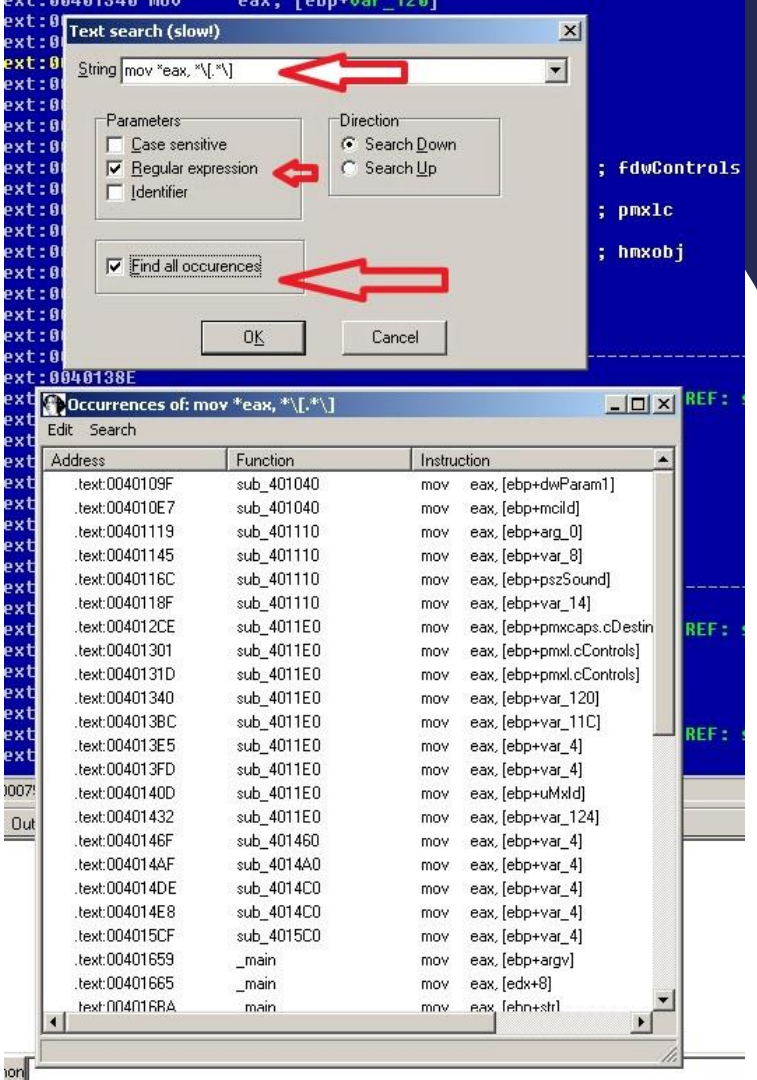
- IDA text searches amount to substring searches through the disassembly listing view.
- Text searches are initiated via **Search->Text**



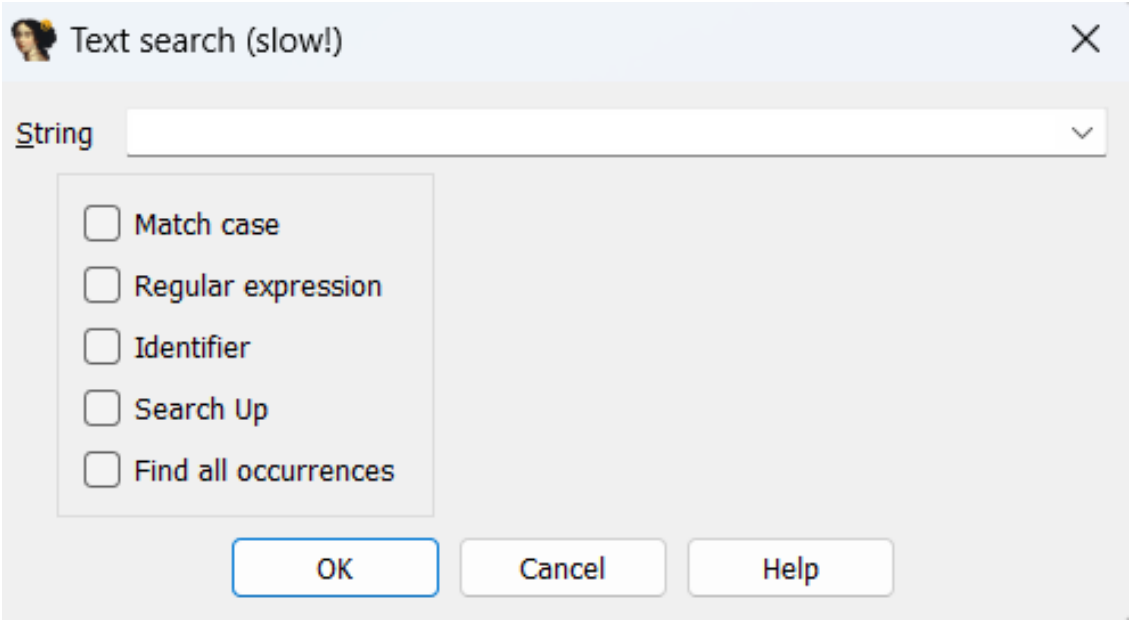
- regular expressions are permitted.
- The Identifier search restricts the search to find whole words only and can match any whole word on an assembly line, including opcode mnemonics or constant values. An Identifier search for 401116 would fail to find a symbol named loc_401116.



One can write a regular expression to find any assignment to the eax register (with the *mov* instruction)



- Selecting **Find all occurrences** causes the search results to be opened in a new window, allowing easy navigation to any single match of the search criteria.
- The previous search can be repeated to locate the next match using **Search->Next Text.**



Binary Searches

- If you need to search for specific binary content such as a known sequence of bytes, then text searches are not the answer.
- While the text search searches the disassembly window, the binary search will search only the content portion of the Hex View window.
- Either the hex dump or the ASCII dump can be searched, depending on how the search string is specified.
- A binary search is initiated using **Search->Sequence of Bytes**

- search string should be specified as a space-separated list of two-digit hex values such as CA FE BA BE, which offers identical behavior as a search for ca fe ba be

